CYBER DECEPTION AI

AI-Based Cyber Deception System for Intelligent Honeypot Placement and Attacker Path Prediction

ARTIFICIAL INTELLIGENCE – PROJECT

Submitted in partial fulfillment of the requirements of the subject

Artificial Intelligence

in

Department of Cybersecurity and Intelligent Systems

by

CHIDWILAS ROUTHU

24BYB0092



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems [SCORE]

VIT, Vellore

October, 2026

DECLARATION

I hereby declare that the Artificial Intelligence project entitled **"Cyber Deception AI"** submitted by me to the School of Computer Science Engineering and Information Systems, VIT, Vellore, is a record of bonafide work carried out by me.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

Chidwilas Routhu (24BYB0092)

CERTIFICATE

This is to certify that the Artificial Intelligence project entitled **"Cyber Deception AI"** submitted by **Chidwilas Routhu (24BYB0092)**, SCORE, VIT, Vellore, is a record of work carried out as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfils the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE Guide

CONTENTS

		Page No.
1	**TITLE**	i
	Declaration	ii
	Certificate	iii
	List of Figures	v
	List of Tables	vi
	List of Abbreviations	vii
2	**ABSTRACT**	1
3	**INTERACTION**	3
4	**LITERATURE SURVEY**	7
5	**SYSTEM ARCHITECTURE**	10
6	**MODULE DESCRIPTION**	13
7	**RESULTS AND IMPLEMENTATIONS**	18
8	**CONCLUSION**	27
9	**REFERENCES**	28
10	**DEPLOYMENT**	30

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Dashboard After an Analysis Run	4
3.2	Network Topology Page With Asset Inspector	5
5.1	System Architecture	10
5.2	Workflow of One Simulation Request	12
7.1	Attack Path Analysis Workspace	18
7.2	Simulation Result and Deception Scoring	19
7.3	Greedy Placement Candidate Scores	20
7.4	Greedy vs Hill Climbing vs Genetic Algorithm	21
7.5	Experiment Lab Strategy Comparison	22
7.6	Advanced Analysis and Defense Planning	22
7.7	Dashboard Before the First Analysis	24
7.8	Stepped Simulation	24
7.9	Saved Scenarios Page	25
7.10	Methodology Page	25
10.1	Live Deployment on GitHub Pages	30

LIST OF TABLES

Table No.	Title	Page No.
4.1	Summary of Surveyed Literature	9
5.1	Built-In Simulated Network	11
7.1	Search Algorithms on the Built-In Network	18
7.2	Greedy Deception Scores	19
7.3	Comparison of the Three Placement Algorithms	20
7.4	Interception Rate by Strategy	21
7.5	Functional Test Cases	23
7.6	Automated Test Results	23
10.1	Deployment Summary	30

LIST OF ABBREVIATIONS

A*	A-star informed search
AI	Artificial Intelligence
API	Application Programming Interface
BFS	Breadth-First Search
CORS	Cross-Origin Resource Sharing
DFS	Depth-First Search
GA	Genetic Algorithm
HC	Hill Climbing
JSON	JavaScript Object Notation
ORM	Object-Relational Mapping
REST	Representational State Transfer
STRIPS	Stanford Research Institute Problem Solver
UCS	Uniform-Cost Search
UI	User Interface

2. ABSTRACT

Honeypots and other deception resources give defenders high-confidence alerts, because no legitimate user should ever interact with them. An organisation cannot place a decoy on every host, however, so the value of deception depends on *where* a limited number of decoys are placed. **Cyber Deception AI** is a simulation and decision-support system that addresses this placement problem on a modelled network.

The network is represented as a weighted, undirected graph in which nodes are hosts and edge weights represent a simulated traversal difficulty. An attacker agent predicts its route from an entry point to a critical asset using four hand-implemented search algorithms: Breadth-First Search, Depth-First Search, Uniform-Cost Search and A*. A defender agent then selects honeypot locations within a fixed budget using three approaches: a transparent **Greedy** deception score, **Hill Climbing** with random restarts, and a **Genetic Algorithm**. All three strategies are evaluated with the same placement-fitness function.

The system is implemented as a full-stack application with a React/TypeScript front end and a FastAPI/NetworkX back end. Saved scenarios are persisted with SQLAlchemy in PostgreSQL. It is publicly deployed on free services: GitHub Pages hosts the front end, and Render hosts the API and database. On the built-in seven-host network, the A* route Internet → Web Server → Application Server → Database has cost 8. With a budget of two honeypots, all three algorithms reach the same fitness (6.2667) and cover all three cheapest attack routes. In a 20-run experiment with one honeypot, random placement intercepts 50% of attacks, while Greedy, Hill Climbing and the Genetic Algorithm intercept 100%.

The experiments are **simulated**, and the network contains a chokepoint: every route passes through the Web Server. The results therefore demonstrate a working, explainable pipeline rather than real-world detection effectiveness. The system uses classical AI techniques (search, heuristics, optimisation, adversarial search and Bayesian reasoning) and contains no trained machine-learning model.

In simple terms: imagine a building with many rooms, one of which holds something valuable. A security team can only afford to place a few hidden cameras. This project helps decide *which rooms* should get the cameras. It does this by first working out the route an intruder would most likely take to reach the valuable room. In the project the “building” is a computer network, the “rooms” are computers, the “valuable room” is a database, and the “cameras” are honeypots: fake computers that raise an alarm when touched.

Keywords: cyber deception, honeypot placement, attack paths, graph search, greedy optimisation, hill climbing, genetic algorithm.

3. INTERACTION

This chapter describes how a user interacts with Cyber Deception AI: the inputs supplied, the processing that each action triggers, and the outputs shown. All behaviour described here was exercised on the running application, both locally and on the public deployment described in Section 10.

The application is designed so that a reviewer can see the whole idea in a few clicks. The user chooses where the attacker starts, what it wants to reach, how the attacker searches, and how many honeypots the defender may place. The system then shows, on a picture of the network, the path the attacker takes, where the honeypots were placed, and whether the attacker was caught. Every number on the screen comes from the back-end algorithms, and every decision comes with a short written explanation.

3.1 Opening the Application

The user opens the deployed site at https://sid07-pro.github.io/cyber-deception-ai/ or, for local use, http://localhost:5173. A left sidebar groups the pages into four sections:

- *Overview:* Dashboard, Network Topology.
- *Analysis:* Attack Path Analysis, Honeypot Placement, Advanced Analysis, Defense Planning.
- *Evaluation:* Experiment Lab, Saved Scenarios.
- *Reference:* Methodology.

The header shows a “Simulated environment” badge. A status indicator polls the real `/api/health` endpoint every 30 seconds and reports the API version and database status (Fig. 3.1).

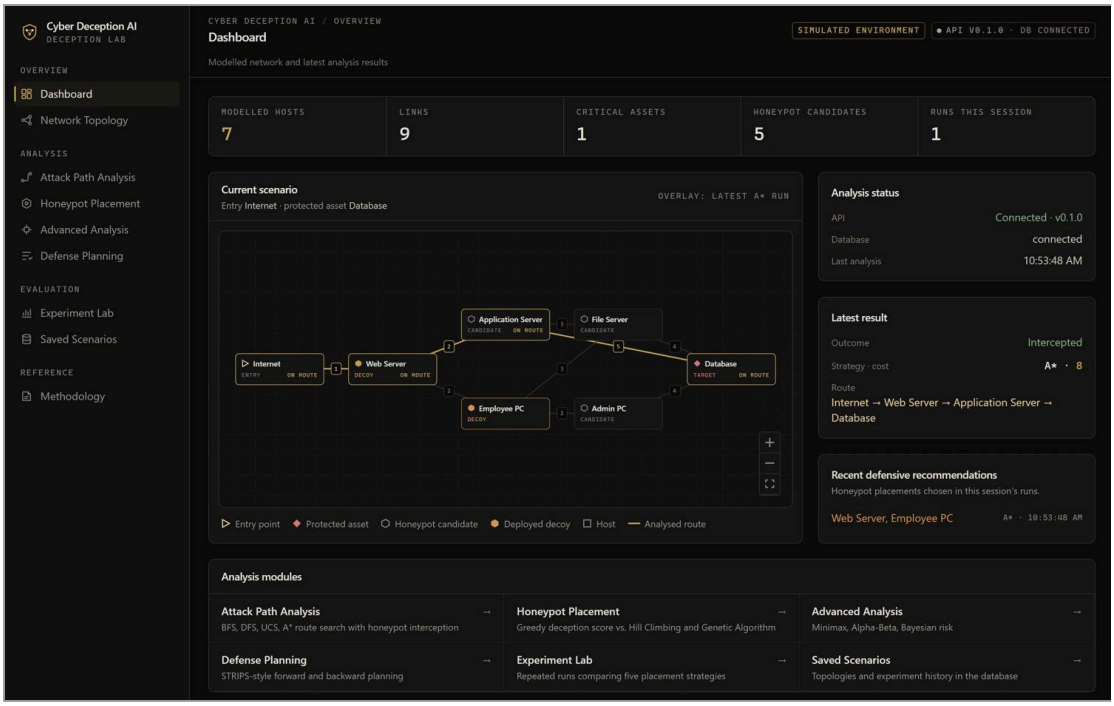


Fig. 3.1 Dashboard After an Analysis Run

3.1.1 A Typical Session, Step by Step

1. Open the website. The Dashboard shows the size of the modelled network: 7 computers, 9 connections, 1 critical asset and 5 possible honeypot locations.
2. Open *Network Topology* and click a computer, for example the Application Server, to see its details and which computers it connects to.
3. Open *Attack Path Analysis*, choose A* as the search strategy, Internet as the start, Database as the target and 2 as the honeypot budget, then press *Run Simulation*.
4. Read the result. The attacker's route is drawn in gold, the two honeypots are marked as decoys, and the panel reports that the attacker was caught at the Web Server on the first step.
5. Open *Honeypot Placement* and press *Evaluate placements* to compare the three optimisation methods on the same network.
6. Open *Experiment Lab* to repeat the experiment many times and compare five strategies, including "no honeypots" and "random honeypots".

3.2 Supplying Network Information

The built-in network is loaded automatically from `GET /api/network`. On the Network Topology page (Fig. 3.2), the user can click any host in the graph or the host table to inspect its type, criticality, degree, deployment cost and links, and can set it as the analysis start or target. Custom topologies can be stored as saved scenarios

through the API and simulated from the Saved Scenarios page. There is no graphical topology editor.

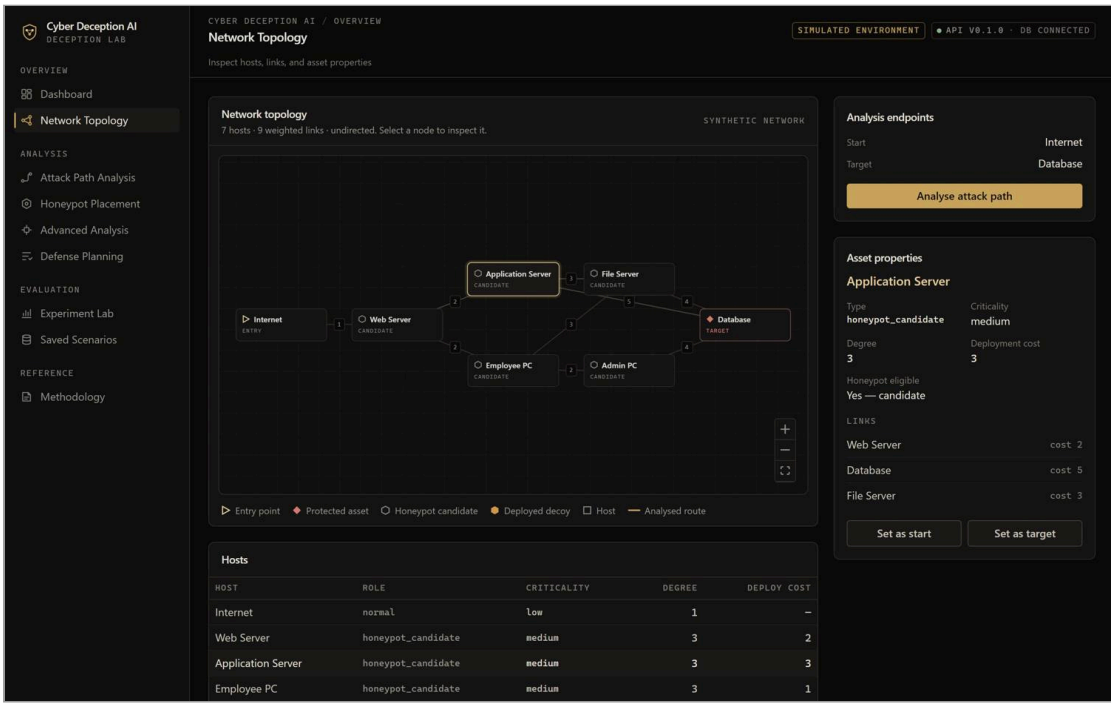


Fig. 3.2 Network Topology Page With Asset Inspector

3.3 Selecting and Executing Algorithms

- **Attack Path Analysis:** the user selects a search strategy (BFS, DFS, UCS or A*), a start node, a target node and a honeypot budget (0–5), then presses *Run Simulation*. A stepped mode moves the attacker one hop at a time.
- **Honeypot Placement:** *Evaluate placements* runs the Greedy score, Hill Climbing and the Genetic Algorithm on the same scenario and budget.
- **Advanced Analysis:** runs Minimax, Alpha-Beta, Bayesian risk, Hill Climbing or the Genetic Algorithm individually.
- **Defense Planning:** generates a forward or backward state-space plan.
- **Experiment Lab:** repeats runs and compares five placement strategies. Results can be exported as JSON or CSV, or saved to the database.

3.4 Presentation of Outputs and Error Handling

Results show the following:

- the predicted route, drawn in gold on the graph;
- the route cost and the number of nodes explored;
- the deployed honeypots, tagged “Decoy”;
- the point where the attacker was intercepted;

- the full deception-score table;
- the three cheapest alternative routes;
- a plain-language explanation of each decision.

Invalid input is rejected at two levels. In the interface, the *Run* and *Evaluate* buttons are disabled when start and target are the same node. On the back end, every request is validated:

- an invalid request body returns HTTP 422;
- an unknown node or scenario returns 404;
- an unavailable database returns 503.

The interface displays the server's message in an error panel.

4. LITERATURE SURVEY

This chapter reviews research on honeypots and cyber deception, game-theoretic defence, graph-based security analysis, search, and the three optimisation approaches compared in this project.

For each area, the chapter explains in simple terms what the earlier work did and how this project uses or differs from it. The aim is to show where the project's ideas come from and what is new in it.

4.1 Honeypots and Cyber Deception

Spitzner [1] established the honeypot as a security resource whose value lies in being probed or attacked. He distinguished low-interaction honeypots, which emulate services, from high-interaction honeypots, which are real monitored systems. Provos [2] introduced Honeyd, a framework that simulates many virtual hosts with configurable network personalities and shows that large decoy networks can be emulated cheaply. Both works focus on building and operating decoys rather than deciding where a limited number should be placed, which is the focus of this project. MITRE Engage [3] frames deception as part of adversary engagement, aligned with defender goals.

What this means for our project: a honeypot is like a fake ATM placed among real ones. Nobody honest has a reason to use it, so anyone who does is suspicious. Earlier work taught us how to *build* such decoys. Our project answers a different question: given only a few decoys, *where* should they go?

4.2 Game-Theoretic and Strategic Deception

Pawlick, Colbert and Zhu [4] survey game-theoretic models of defensive deception and classify them into six categories: perturbation, moving-target defence, obfuscation, mixing, honey-x and attacker engagement. Tambe [5] describes security games in which a defender with limited resources commits to a strategy and an attacker best-responds. This project borrows the attacker–defender framing but implements depth-limited adversarial search (Minimax and Alpha-Beta [6]) with a hand-designed utility. It does not compute a game-theoretic equilibrium.

What this means for our project: game theory treats security as a contest between two sides that each try to do their best. Our project uses the same two-player idea in a simpler form. The attacker moves, the defender responds, and a search looks a few moves ahead to choose the best response, much like a computer playing chess.

4.3 Graph-Based Network Security Analysis

Sheyner et al. [7] showed that attack graphs can be generated automatically from a network model and used to find the paths that reach a goal. Ou, Govindavajhala and Appel [8] proposed MulVAL, a logic-based analyser that derives attack graphs from vulnerability and configuration data. Durkota et al. [9] combined attack graphs with game theory to compute defensive hardening, including honeypot deployment. Cyber Deception AI uses a simpler weighted host graph without vulnerability data. Its contribution is to connect route prediction directly to decoy placement in an explainable, interactive tool.

What this means for our project: an attack graph is a map of how an attacker could move from one computer to another. Our project builds a simple version of this map, in which computers are points and connections are lines, and then searches it for the attacker's most likely route.

4.4 Search Algorithms

Dijkstra's algorithm [10] computes minimum-cost paths in graphs with non-negative weights; Uniform-Cost Search is its single-target form. Hart, Nilsson and Raphael [11] introduced A*, which orders expansion by f(n) = g(n) + h(n) and is optimal when the heuristic h is admissible. Yen [12] gave an algorithm for the K shortest loopless paths, which this project uses through NetworkX [13] to enumerate alternative attack routes. Standard treatments appear in Cormen et al. [14] and Russell and Norvig [15].

What this means for our project: these algorithms are different ways of finding a route on a map. Some count the number of steps (BFS), some explore one branch at a time (DFS), and some add up the "difficulty" of each step to find the easiest overall route (UCS and A*). Our attacker uses them to choose its path.

4.5 Greedy Optimisation, Hill Climbing and Genetic Algorithms

Nemhauser, Wolsey and Fisher [16] proved that greedy selection achieves a (1 − 1/e) approximation when maximising a monotone submodular set function, such as coverage, under a cardinality constraint. That guarantee applies to marginal-gain greedy selection on a submodular objective. This project's greedy method instead ranks nodes by an independent per-node score, so the guarantee does not transfer. Hill Climbing is a local-search method that is prone to local optima, a weakness that random restarts mitigate [15]. Genetic Algorithms, introduced by Holland [17] and popularised by Goldberg [18], evolve a population of solutions through selection, crossover and mutation. STRIPS [19] provides the action representation used by the planning module.

What this means for our project: choosing honeypot locations is an *optimisation* problem, meaning we want the best choice from many possible choices. A **greedy** method picks whatever looks best right now. **Hill Climbing** starts with a guess and keeps making small improvements, like walking uphill until no step goes higher. A **Genetic Algorithm** copies natural evolution: it keeps a "population" of guesses, combines good ones and makes small random changes. The project compares all three on the same problem.

Table 4.1 Summary of Surveyed Literature

Ref.	Problem / approach	Relevance and difference
[1], [2]	Honeypot design; virtual honeypot framework	Defines the decoy concept; does not address budgeted placement
[4], [5]	Game-theoretic deception; security games	Attacker–defender framing; this project uses adversarial search, not equilibria
[7]–[9]	Attack graphs; logic-based analysis; attack-graph games	Route modelling; this project uses a simpler weighted host graph
[10]–[12]	Shortest paths; A*; K shortest loopless paths	Directly implemented for attacker routes and alternatives
[16]–[18]	Greedy approximation; genetic algorithms	Basis of the placement methods; [16]'s guarantee does not apply to the per-node score

5. SYSTEM ARCHITECTURE

The application is a three-tier web system (Fig. 5.1):

- a React single-page application calls a FastAPI REST service over JSON;
- the service layer composes framework-independent algorithm modules written in Python with NetworkX;
- SQLAlchemy persists saved scenarios and experiment results.

In simple terms, the system has three layers that work like a restaurant. The **front end** is the dining room: what the user sees and clicks. The **back end** is the kitchen: it receives each request, runs the algorithms and sends back the result. The **database** is the store room: it keeps saved networks and experiment results so they are not lost. The layers talk to each other through a well-defined set of requests called an API.

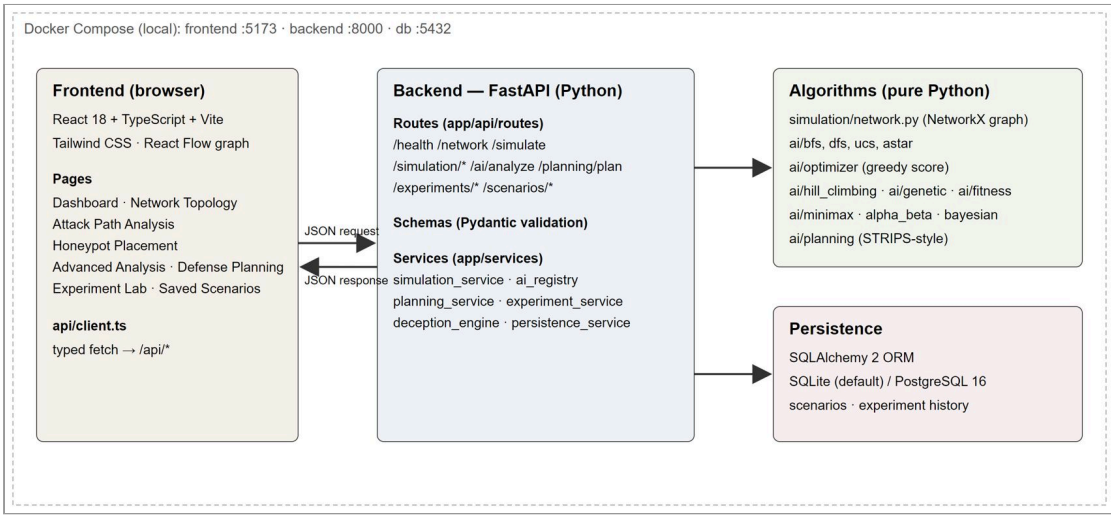


Fig. 5.1 System Architecture

5.1 Components

- **Front end** (`frontend/src`): React 18, TypeScript, Vite 5, Tailwind CSS and React Flow 11. A typed client issues all API requests.
- **API layer** (`backend/app/api/routes`): handles routing, status codes and error mapping. Pydantic schemas validate every request.
- **Service layer** (`backend/app/services`): the simulation, AI registry, planning, experiment, deception-engine and persistence services. They contain no HTTP code.
- **Algorithm layer** (`backend/ai`, `backend/simulation`): graph model, search, placement, game search, Bayesian reasoning and planning.

- **Persistence**: SQLAlchemy 2 ORM. It uses SQLite by default and PostgreSQL under Docker and on Render.

A key design decision is that the algorithm code does not depend on the web framework or the database. This keeps each part simple, lets the algorithms be tested on their own, and means the same algorithm code is used by the simulation page, the experiments and the automated tests.

5.2 Graph Representation

The built-in network (`backend/simulation/network.py`) is an undirected NetworkX graph G = (V, E) with |V| = 7 and |E| = 9:

- each node carries a role, and each edge carries a weight;
- Internet is the entry point and Database is the critical target;
- the other five hosts are honeypot candidates.

All weights are hand-chosen simulation values (Table 5.1). Internet has a single neighbour, Web Server, so every attack route passes through Web Server.

A **graph** is simply a set of points (called nodes) joined by lines (called edges). Here each node is a computer and each edge is a network connection. Each edge carries a number, its *weight*, which represents how difficult that step is for an attacker. A higher number means a harder step. The total difficulty of a route is the sum of the weights along it. For example, the route Internet → Web Server → Application Server → Database costs 1 + 2 + 5 = 8.

Table 5.1 Built-In Simulated Network

Edge	Weight	Edge	Weight
Internet – Web Server	1	Employee PC – Admin PC	2
Web Server – Application Server	2	Admin PC – Database	4
Web Server – Employee PC	2	File Server – Database	4
Application Server – Database	5	Employee PC – File Server	3
Application Server – File Server	3		

5.3 Data Flow

Fig. 5.2 shows how one simulation request is processed. The request moves from user input through validation, graph construction, attacker search, defender placement and evaluation, and finally to the response rendered by the interface.

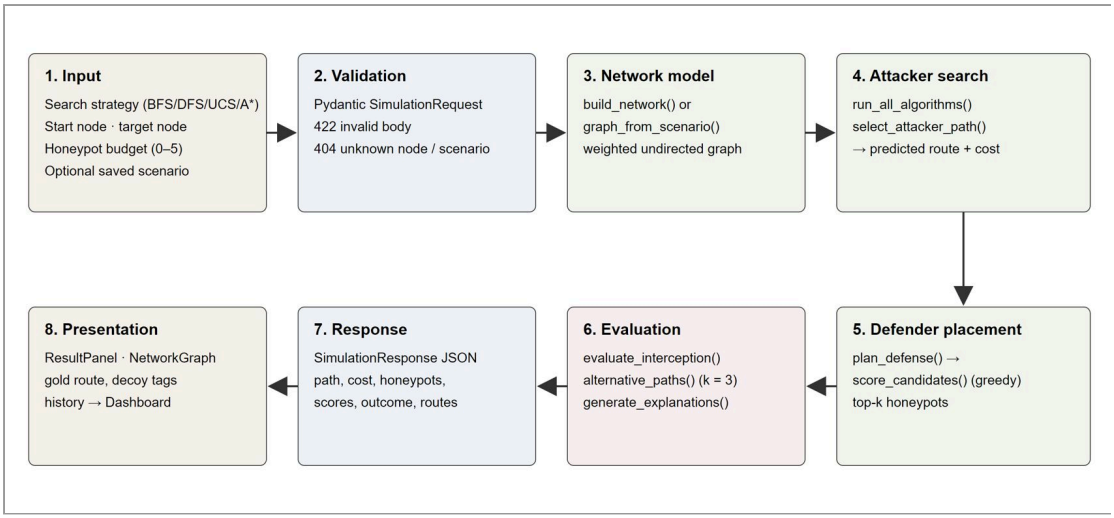


Fig. 5.2 Workflow of One Simulation Request

5.4 Deployment Architecture

The production deployment has three parts:

- the static front end, built by a GitHub Actions workflow and served by GitHub Pages;
- the FastAPI service, run by Render as a free web service;
- a free Render PostgreSQL database for persistence.

Locally, Docker Compose runs the same components as three services: `db`, `backend` and `frontend`.

6. MODULE DESCRIPTION

This chapter describes each implemented module, giving its purpose, inputs, processing, outputs and source files.

For each module, a short plain-language explanation is given first, followed by the technical details. Worked examples use the real values from the built-in network.

6.1 Network Model

In simple terms: this module draws the map. It knows which computers exist, which are connected, how hard each connection is to cross, which computer is the attacker's entry point and which one must be protected.

`build_network()` adds nodes with roles and weighted edges. `graph_from_scenario()` rebuilds a graph from stored nodes and edges and reads optional per-node deployment costs. Saved topologies are validated on save: node IDs must be unique, edge endpoints must exist, weights must be positive, and the start and target must be distinct nodes.

6.2 Attack-Route Prediction

In simple terms: this module plays the role of the attacker. Given the map, it searches for a way from the Internet to the Database. Different search strategies find routes in different ways, and the module reports which route each strategy finds, how difficult that route is, and how much searching was needed.

Four hand-written searches share the interface `search(graph, start, goal) → (path, cost, nodes_explored)`:

- **BFS:** uses a FIFO queue and finds the fewest hops. Time O(V + E).
- **DFS:** uses an explicit stack and gives no optimality guarantee. Time O(V + E).
- **UCS:** uses a min-priority queue on g(n) and is optimal for positive weights. Time O((V + E) log V).
- **A*:** uses a priority queue on f(n) = g(n) + h(n), where h(n) is the hop distance to the target multiplied by the minimum edge weight. This heuristic is admissible, so A* is optimal.

`run_all_algorithms()` runs all four searches and supplies their routes to the defender; `select_attacker_path()` then commits to the chosen strategy's route. Alternative routes are the three cheapest loop-free paths from Yen's algorithm [12], computed by `alternative_paths()`.

Worked example (BFS versus UCS): suppose there are two ways to reach a target. Route A has 2 steps of difficulty 5 each (total 10), and Route B has 3 steps of

difficulty 1 each (total 3). BFS counts steps, so it picks Route A. UCS adds up difficulty, so it picks Route B, the easier route. This exact case is one of the project's automated tests. On the built-in network, all four strategies happen to find the same route, with cost 8.

6.3 Greedy Placement

In simple terms: the greedy method gives every possible honeypot location a score and picks the highest scores. A location scores well if attack routes pass through it, if it is close to the Database, and if it is well connected, but the score is divided by the cost of putting a honeypot there. It is like choosing where to put a security guard: a busy junction near the vault is a good spot, but an expensive spot needs to be worth its price.

The greedy method (`backend/ai/optimizer.py: score_candidates`) scores each candidate n independently, excluding the start and target, and selects the top k by Eqn. 6.1:

$$score(n) = [\, freq(n) + 1 / (hops(n, target) + 1) + deg(n) \,] / cost(n) \quad (6.1)$$

The terms in Eqn. 6.1 are defined as follows:

- **freq(n):** the number of candidate routes that contain n;
- **hops(n, target):** the unweighted distance from n to the target;
- **deg(n):** the node degree;
- **cost(n):** the simulated deployment cost (Web Server 2, Application Server 3, Employee PC 1, Admin PC 2, File Server 2).

The method is fast and transparent, but it ignores interactions between the selected decoys.

Worked example: for the Web Server, 4 of the 4 predicted routes pass through it (freq = 4), it is 2 steps from the Database (proximity = 1/(2+1) = 0.333), it has 3 connections (deg = 3), and its cost is 2. Its score is therefore (4 + 0.333 + 3) / 2 = 3.667, the highest of all candidates, so it is chosen first.

6.4 Placement Fitness

In simple terms: the greedy score judges one location at a time, but a defender really places a *group* of honeypots. The fitness function gives one overall mark to a whole group. Its most important part is *coverage*: how many of the attacker's possible routes would hit at least one honeypot. A fair comparison needs one common marking scheme, so the same fitness function is used to judge every method.

`backend/ai/fitness.py: evaluate()` scores a whole placement P using Eqn. 6.2:

$$F(P) = 4 \cdot coverage + 2 \cdot proximity + 1 \cdot criticality + 1.5 \cdot deception - 1 \cdot cost_{norm} \quad (6.2)$$

The terms in Eqn. 6.2 are defined as follows:

- **coverage:** the fraction of candidate routes that contain at least one node of P;
- **proximity:** the mean of 1/(1 + hops to target);
- **criticality:** the mean role weight;
- **deception:** the mean of degree divided by maximum degree;
- **$cost_{norm}$:** the normalised deployment cost.

Hill Climbing and the Genetic Algorithm maximise F, and the experiment service reports F for every strategy, so all strategies are compared on one objective.

6.5 Hill Climbing

In simple terms: Hill Climbing starts with one choice of honeypot locations and tries small changes, such as swapping one location for another, adding one or removing one. Whenever a change gives a better fitness mark, it keeps the change and continues. When no small change helps any more, it stops. Because it can get stuck on a "small hill", it starts again from four different starting points and keeps the best result.

`backend/ai/hill_climbing.py` performs hill climbing over placements of at most k eligible nodes. A placement's neighbours are those one swap, drop or add away. There are four restarts: the first starts from the highest-degree hosts and the others from random placements produced by a seeded generator. Each pass scans the neighbours in a deterministic order and accepts any improving neighbour as soon as it is found. The search stops when a full pass brings no improvement.

```
best ← None
for restart in 1..4:
    P ← highest-degree hosts (first restart) or random placement
    repeat:
        improved ← false
        for N in neighbours(P):            # swap, add or drop one
node
            if F(N) > F(P): P ← N; improved ← true
    until not improved
    best ← better of (best, P)
return best
```

6.6 Genetic Algorithm

In simple terms: the Genetic Algorithm keeps 12 candidate placements at once, like a population of individuals. In each of 15 generations it keeps the stronger candidates (selection), mixes two of them to make a new one (crossover), and occasionally changes one location at random (mutation). Over the generations the population tends to improve. Because the random choices use a fixed seed, the same run always gives the same answer, which makes the results repeatable.

`backend/ai/genetic.py` represents each individual as a set of exactly k eligible nodes. Its parameters are:

- population 12, evolved for 15 generations;
- tournament selection;
- crossover that mixes the two parents' nodes;
- mutation rate 0.2;
- a repair step that restores exactly k distinct nodes.

All randomness is seeded, so runs are reproducible.

6.7 Interception and Evaluation

In simple terms: once honeypots are placed, the system walks the attacker along its route, one step at a time. If the attacker reaches a honeypot before the Database, the attack is counted as *intercepted*, which means the defender wins. The experiment module repeats this many times and reports how often each strategy catches the attacker.

`evaluate_interception()` returns the first honeypot on the attacker path and its position. `experiment_service.run_experiment()` repeats runs for five strategies (none, random, greedy, hill climbing and genetic algorithm). It aggregates the interception rate, attack success rate, path cost, steps, fitness and execution time.

6.8 Supporting Modules

- **Adversarial search** (`deception_game.py, minimax.py, alpha_beta.py`): MAX (the attacker) moves one hop; MIN (the defender) places a honeypot or passes. The terminal utility is ±50.
- **Bayesian risk** (`bayesian.py`): sequential Bayes updates using assumed likelihoods.
- **Planning** (`planning.py`): STRIPS-style forward search and backward regression, both breadth-first over states.
- **Back-end API and persistence**: health, network, simulation, session, AI, planning, experiment and scenario endpoints, with SQLAlchemy models for

scenarios and experiment records.

7. RESULTS AND IMPLEMENTATIONS

All results were produced by the project's own code on the built-in **simulated** network. The screenshots are genuine captures of the running application. Runtimes are machine-dependent and are reported only as an indication.

Each result is followed by a short explanation of what it shows. The three questions answered are: which route does the attacker take, where should the honeypots go, and how do the three optimisation methods compare?

7.1 Attack-Route Prediction

Table 7.1 Search Algorithms on the Built-In Network

Algorithm	Predicted route (Internet → Database)	Cost	Nodes explored
BFS	Internet → Web Server → Application Server → Database	8	5
DFS	same	8	4
UCS	same	8	7
A*	same	8	7

All four algorithms return the same route (Table 7.1). A* explores as many nodes as UCS because its admissible heuristic is weak on this graph. The three cheapest loop-free routes cost 8, 9 and 10 (Fig. 7.1, Fig. 7.2).

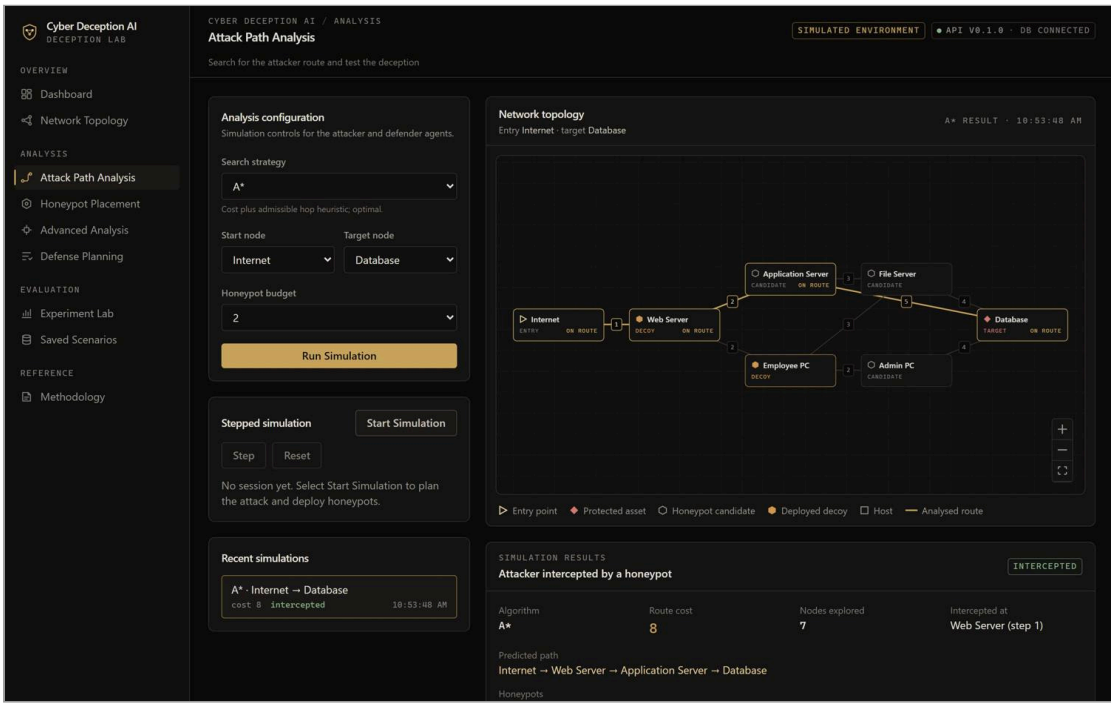


Fig. 7.1 Attack Path Analysis Workspace

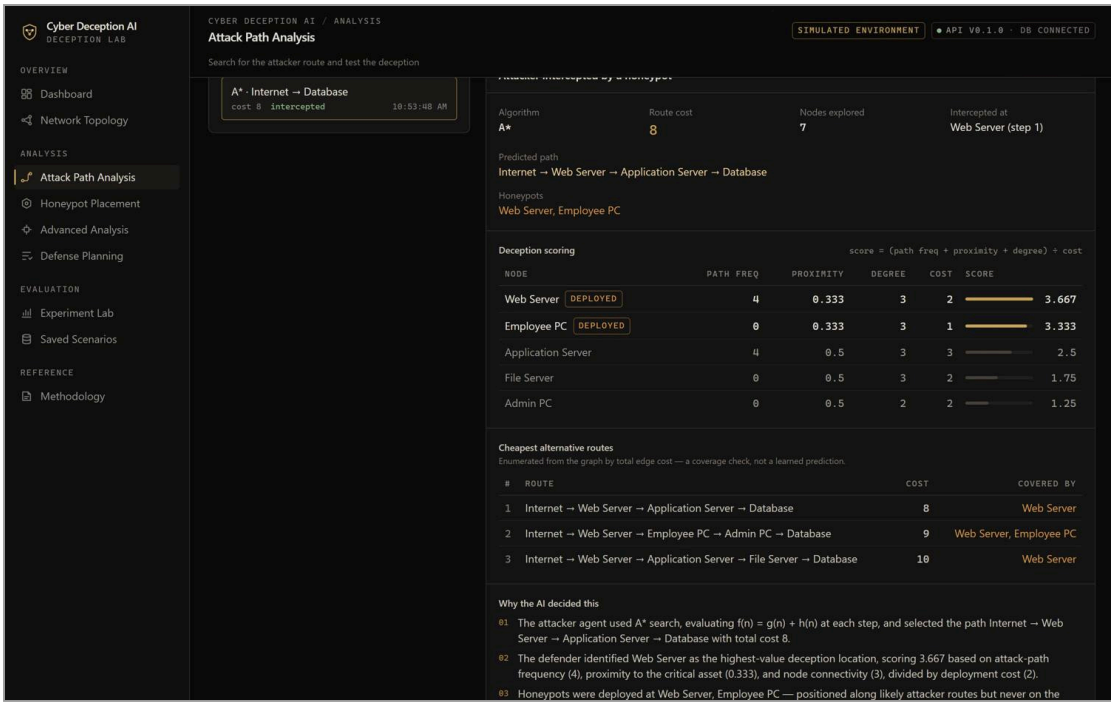


Fig. 7.2 Simulation Result and Deception Scoring

What this shows: every strategy agrees that the easiest way to the Database is through the Web Server and the Application Server, with a total difficulty of 8. The strategies differ only in how much searching they needed, from 4 to 7 computers examined.

7.2 Greedy Placement

Table 7.2 Greedy Deception Scores (A*, Budget k = 2)

Candidate	freq	proximity	degree	cost	score	Selected
Web Server	4	0.333	3	2	3.667	Yes
Employee PC	0	0.333	3	1	3.333	Yes
Application Server	4	0.5	3	3	2.5	No
File Server	0	0.5	3	2	1.75	No
Admin PC	0	0.5	2	2	1.25	No

By Eqn. 6.1, score(Web Server) = (4 + 1/3 + 3) / 2 = 3.667. Employee PC is selected mainly because of its low cost and degree, even though no predicted route uses it (Fig. 7.3).

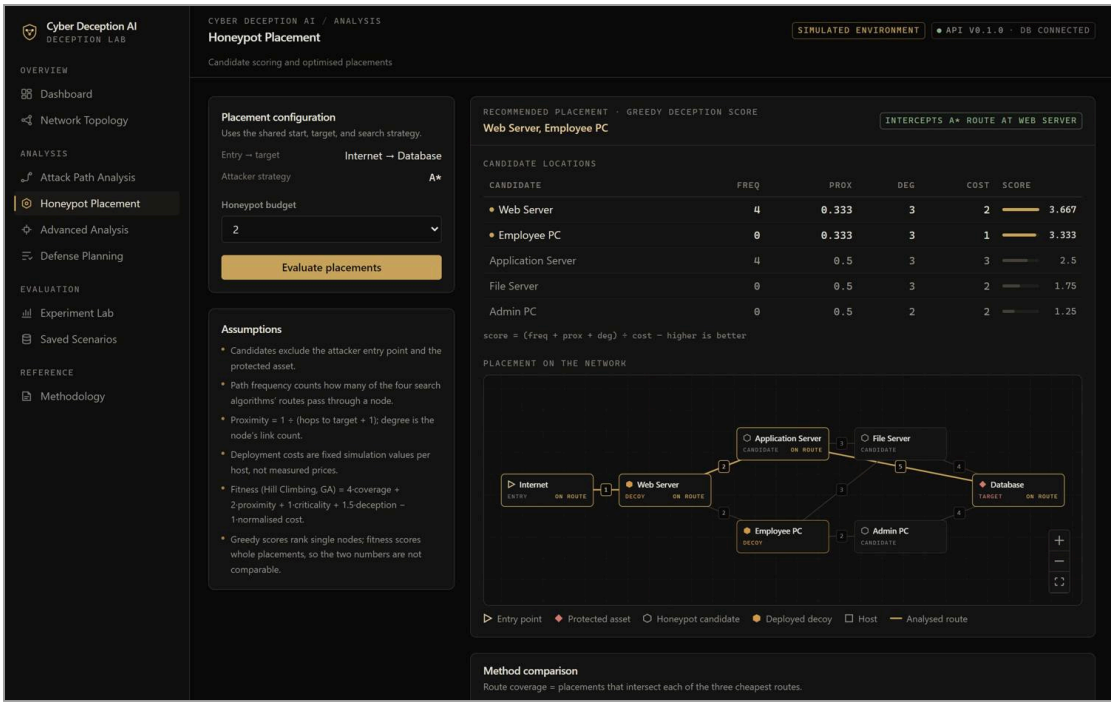


Fig. 7.3 Greedy Placement Candidate Scores

What this shows: the Web Server is the most valuable place for a honeypot, because every attack route passes through it. This matches intuition, since it is the only "door" from the Internet into the network.

7.3 Comparison of Greedy, Hill Climbing and Genetic Algorithm

The three algorithms were run on the same network, attacker strategy (A*), candidate set and budget (k = 2). Routes covered counts how many of the three cheapest routes intersect the placement. Fitness is given by Eqn. 6.2, and runtime is the average per run over 20 runs.

Table 7.3 Comparison of the Three Placement Algorithms

Algorithm	Placement selected	Routes covered	Coverage	Fitness	Evaluations	Avg. runtime
Greedy	Web Server, Employee PC	3 / 3	100%	6.2667	—	0.13 ms
Hill Climbing	Application Server, Employee PC	3 / 3	100%	6.2667	44	2.28 ms
Genetic Algorithm	File Server, Web Server	3 / 3	100%	6.2667	194	13.16 ms

All three methods reach the same fitness with different placements, which shows that the objective has ties on this small network (Table 7.3, Fig. 7.4). Greedy is

the fastest, but its advantage here comes from the chokepoint rather than from a general guarantee.

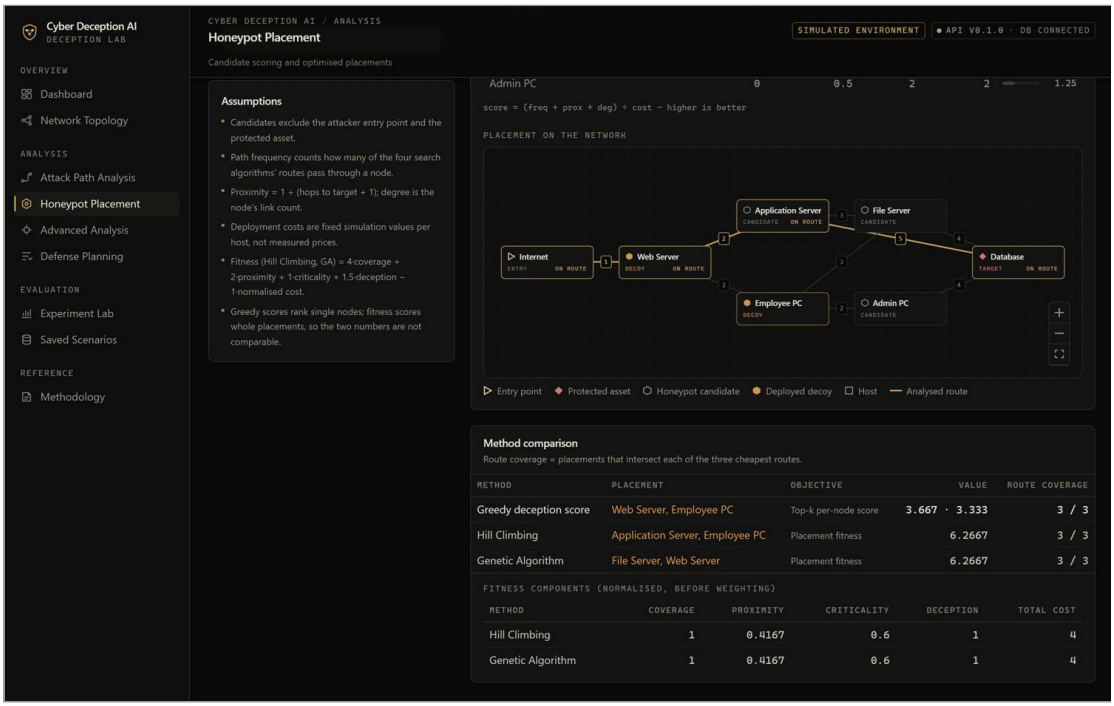


Fig. 7.4 Greedy vs Hill Climbing vs Genetic Algorithm

What this shows: the three methods chose different pairs of computers, but all reached the same fitness mark and all covered every attack route. This means several equally good answers exist on this small network. Greedy found its answer fastest (about 0.13 ms), while the Genetic Algorithm took longest (about 13 ms) because it evaluates many more candidates.

7.4 Interception Experiment

Table 7.4 Interception Rate by Strategy (A*, 20 Runs, Seed 0)

Strategy	Budget 1	Budget 2	Avg. fitness (budget 1)
No deception	0%	0%	0.0
Random placement	50%	85%	4.1917
Greedy	100%	100%	6.1
Hill Climbing	100%	100%	6.1
Genetic Algorithm	100%	100%	6.1

The attacker is deterministic, so only the random strategy varies between runs. The 100% interception rate follows from the chokepoint at Web Server and is not a real-world detection rate (Table 7.4, Fig. 7.5).

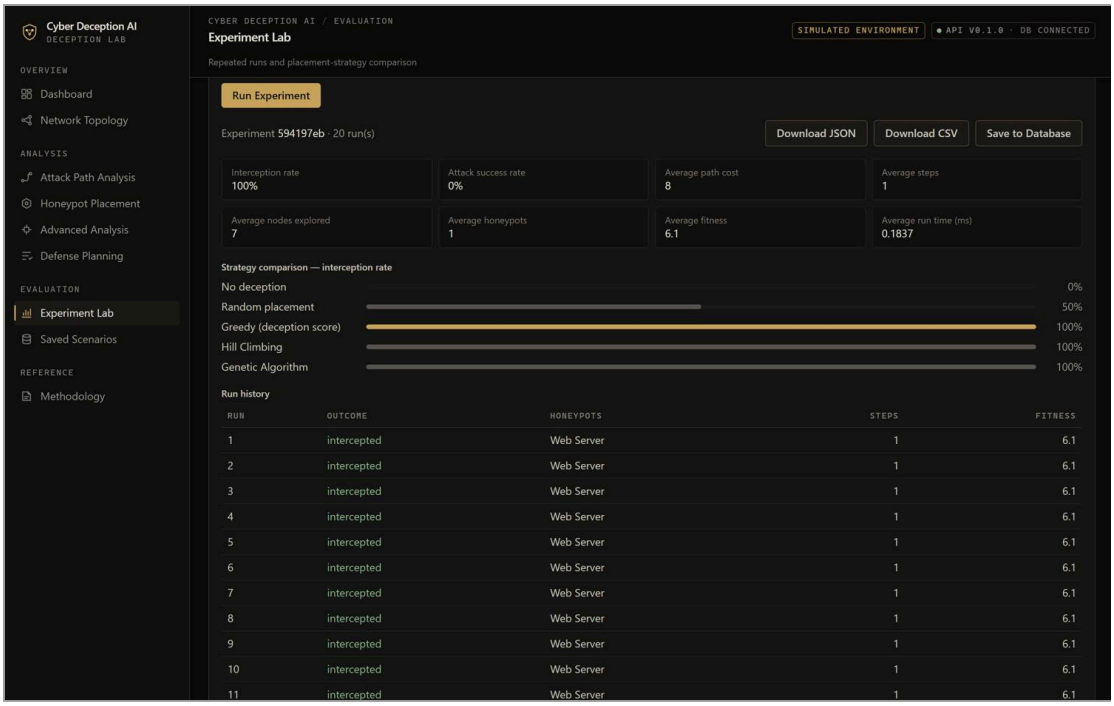


Fig. 7.5 Experiment Lab Strategy Comparison

What this shows: with no honeypots the attacker always reaches the Database. Placing honeypots at random catches it only half of the time with one honeypot. All three intelligent methods catch it every time. This demonstrates the value of choosing honeypot locations carefully rather than at random.

7.5 Supporting Analyses

At depth 4, Minimax visited 69 states and Alpha-Beta 35 (14 branches pruned); both chose “move Web Server” with value 2.6667. The Bayesian module gives the Database a posterior of 0.969 from a prior of 0.70, based on assumed likelihoods. Forward planning (48 states explored) and backward regression (5 states) both found the same three-action plan, with cost 7 (Fig. 7.6).

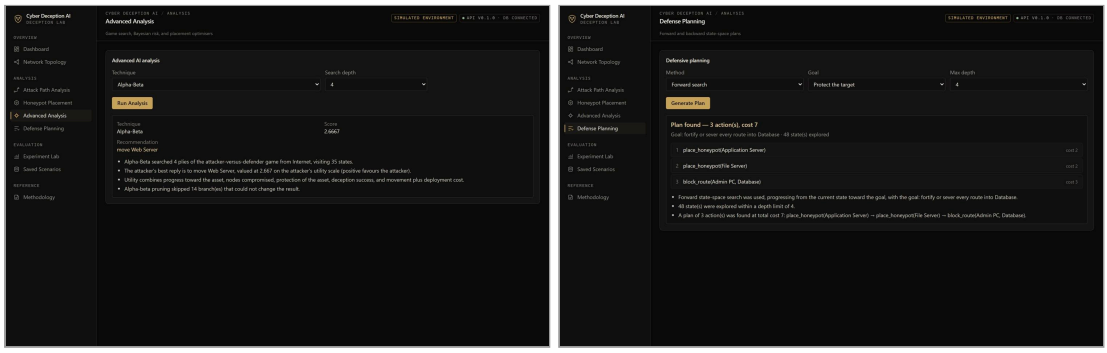


Fig. 7.6 Advanced Analysis (Alpha-Beta) and Defense Planning

7.6 Testing

Table 7.5 Functional Test Cases

#	Input	Expected behaviour	Actual result	Status
1	POST /api/simulate {algorithm: "UCS"}	Cheapest route, honeypots placed	Cost 8; Web Server, Employee PC; intercepted	Pass
2	honeypot_count = 99	Validation error	HTTP 422	Pass
3	start_node = "Mainframe"	Unknown-node error	HTTP 404 with known nodes	Pass
4	Disconnected target (test graph)	No path	Empty path, outcome no_path	Pass
5	Simulate saved scenario	Uses stored topology	HTTP 200, scenario_id returned	Pass
6	scenario_id = 999	Not found	HTTP 404	Pass
7	Stepped session, one step	Attacker moves one hop	At Web Server, intercepted, step 1	Pass
8	Start = target in UI	Action blocked	Buttons disabled with message	Pass
9	GET /api/health on the deployed API	API and database status	status ok, database connected	Pass

Table 7.6 Automated Test Results (27 October 2026)

Suite	Command	Result
Back end (pytest, 12 files)	`python -m pytest -q`	170 passed
Front end (Vitest, 9 files)	`npx vitest run`	44 passed
Production build	`npm run build`	Succeeded

7.7 Implementation Screenshots

The following screenshots, captured from the deployed application, show the remaining implemented screens.

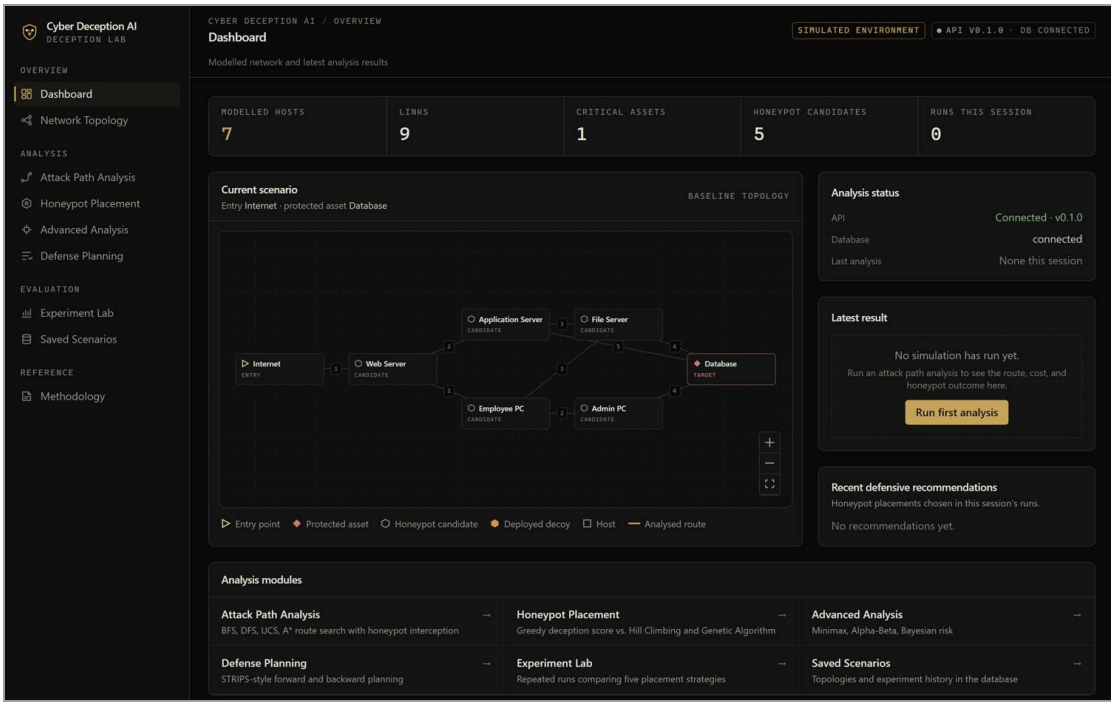


Fig. 7.7 Dashboard Before the First Analysis (Empty State)

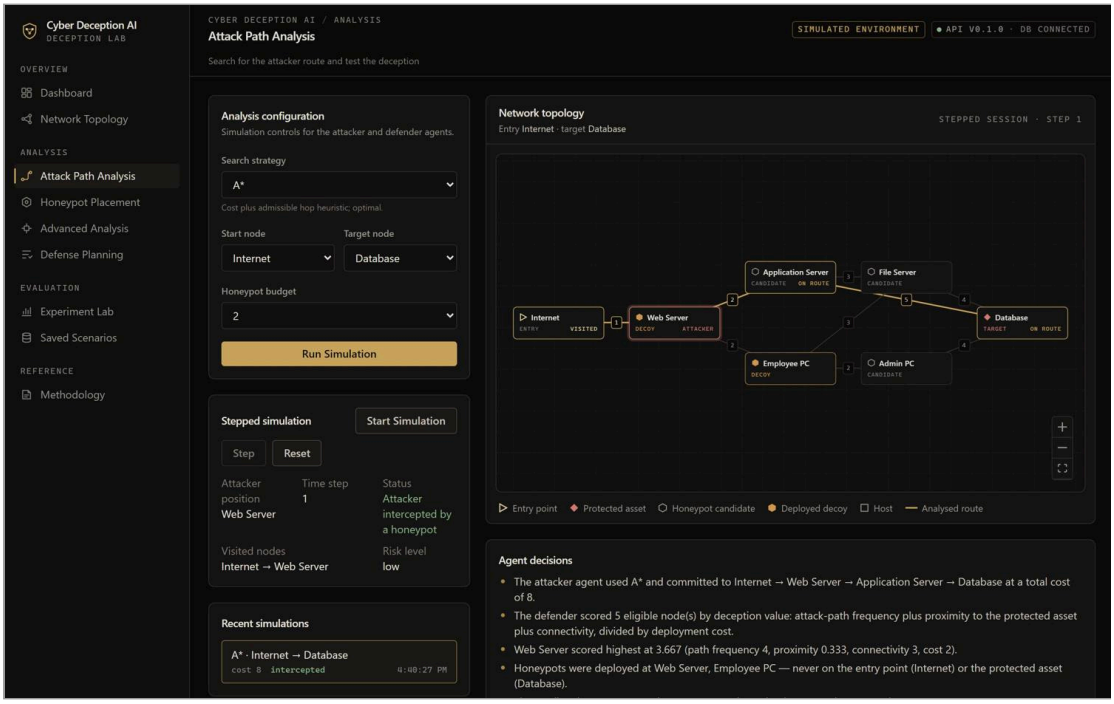


Fig. 7.8 Stepped Simulation: Attacker Intercepted at Web Server on Step 1

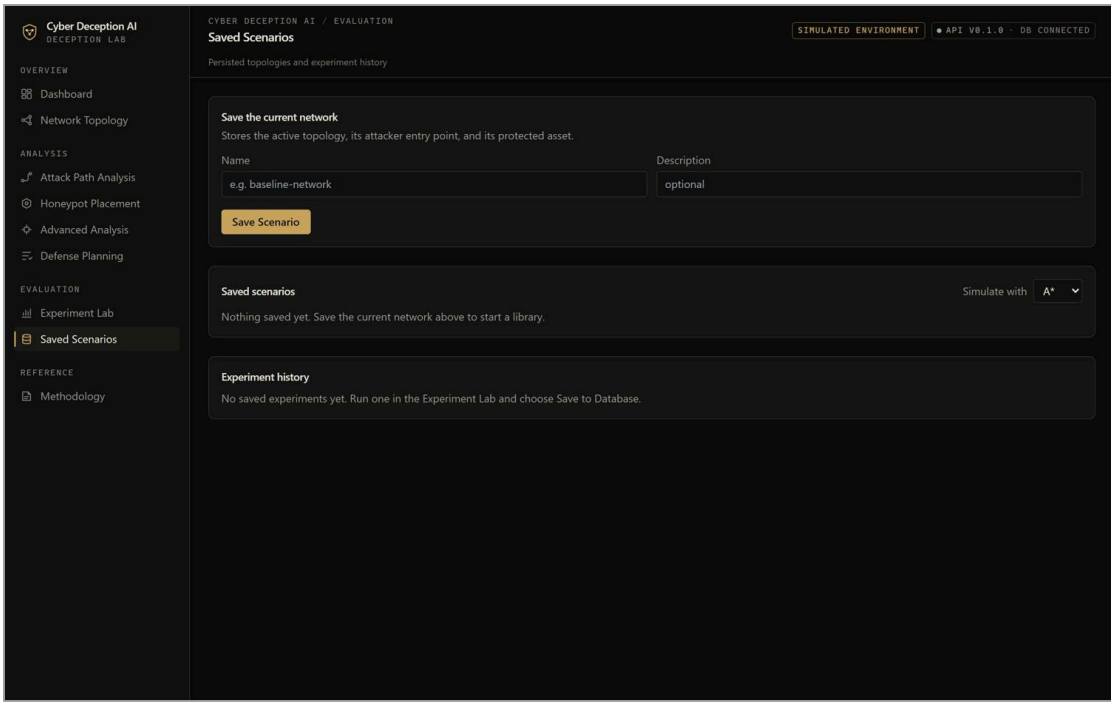


Fig. 7.9 Saved Scenarios Page Backed by the Database

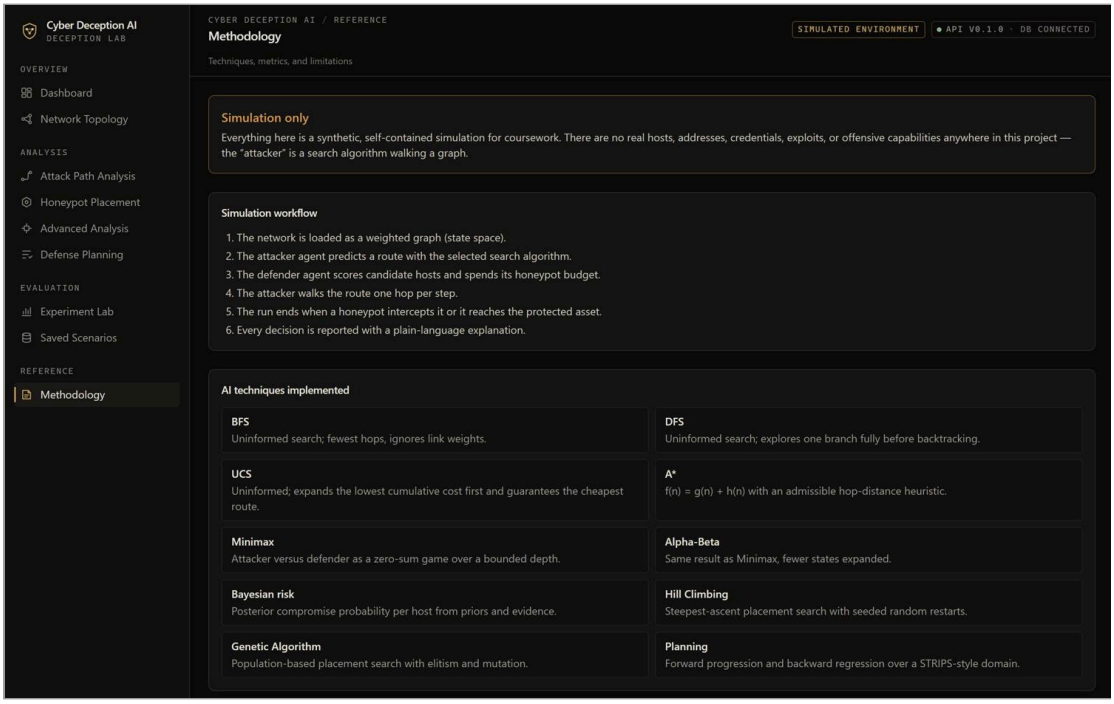


Fig. 7.10 Methodology Page

7.8 Key Source Code

The core loop of Uniform-Cost Search (`backend/ai/ucs.py`) pops the cheapest frontier entry and returns only when the goal is popped. This is what guarantees optimality:

```
while frontier:
    cost, _, current, path = heapq.heappop(frontier)
    if current in visited:
        continue
    visited.add(current)
    nodes_explored += 1
    if current == goal:
        return path, cost, nodes_explored
    for neighbor in graph.neighbors(current):
        edge_cost = graph[current][neighbor]["weight"]
        new_cost = cost + edge_cost
        if neighbor not in best_cost or new_cost <
best_cost[neighbor]:
            best_cost[neighbor] = new_cost
            counter += 1
            heapq.heappush(frontier, (new_cost, counter, neighbor,
path + [neighbor]))
return [], 0, nodes_explored
```

The greedy deception score (`backend/ai/optimizer.py`) implements Eqn. 6.1 and takes the top k candidates:

```
for node in candidates:
    freq = _attack_path_frequency(node, candidate_paths)
    proximity = round(_target_proximity(graph, node, target), 3)
    connectivity = _node_connectivity(graph, node)
    cost = (DEPLOYMENT_COST if deployment_costs is None else
deployment_costs).get(node, 1)
    raw_score = (freq + proximity + connectivity) / cost
    scored.append({"node": node, ..., "deception_score":
round(raw_score, 3)})
scored.sort(key=lambda x: x["deception_score"], reverse=True)
selected = [item["node"] for item in scored[:top_n]]
return scored, selected
```

7.9 Limitations

- A single seven-host synthetic topology with hand-set weights, costs and probabilities; all experiments are simulated.
- The attacker is assumed rational, cost-minimising and fully informed, and does not detect decoys.
- All four searches return the same route here, so the fitness route set contains one distinct route.
- The per-node greedy score ignores redundancy between decoys.
- Honeypot false positives and decoy isolation are not modelled.

8. CONCLUSION

This project addressed the problem of placing a limited number of honeypots in a network. It modelled the network as a weighted graph and predicted attacker routes with BFS, DFS, UCS and A*. It then compared three placement approaches on a common objective: a transparent Greedy score, Hill Climbing and a Genetic Algorithm. The work was delivered as a tested full-stack application with an interactive graph, explanations for every decision, an experiment laboratory and persistence. The application is publicly deployed on free hosting.

On the simulated network, all three algorithms reached the same fitness (6.2667) and covered all three cheapest routes with two honeypots. They intercepted the attacker in every run, compared with 50% (budget 1) and 85% (budget 2) for random placement, and Greedy was the fastest. These findings are limited by the network's chokepoint and by the simplified attacker model.

In simple terms: the project shows that a computer can be used to decide sensibly where to place a small number of decoys in a network. It does this by first predicting the attacker's route and then choosing locations that block that route. It also shows, in a way anyone can check, that careful placement works much better than random placement on the tested network.

The main contribution is an explainable pipeline in which the defender's decision is computed directly from the attacker's predicted routes. Future work includes:

- larger, generated topologies without chokepoints;
- directed, vulnerability-aware graphs;
- dynamic attacker models;
- marginal-gain greedy and exact set-cover placement;
- game-theoretic defence;
- broader evaluation metrics with confidence intervals.

9. REFERENCES

1. L. Spitzner, *Honeypots: Tracking Hackers*. Boston, MA, USA: Addison-Wesley, 2003.
2. N. Provos, “A virtual honeypot framework,” in *Proc. 13th USENIX Security Symposium*, San Diego, CA, USA, 2004.
3. The MITRE Corporation, “MITRE Engage.” [Online]. Available: https://engage.mitre.org
4. J. Pawlick, E. Colbert, and Q. Zhu, “A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy,” *ACM Computing Surveys*, vol. 52, no. 4, Art. 82, 2019, doi: 10.1145/3337772.
5. M. Tambe, *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge, U.K.: Cambridge University Press, 2011.
6. D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.
7. O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated generation and analysis of attack graphs,” in *Proc. IEEE Symposium on Security and Privacy*, 2002, pp. 254–265, doi: 10.1109/SECPRI.2002.1004377.
8. X. Ou, S. Govindavajhala, and A. W. Appel, “MulVAL: A logic-based network security analyzer,” in *Proc. 14th USENIX Security Symposium*, Baltimore, MD, USA, 2005.
9. K. Durkota, V. Lisý, B. Bošanský, and C. Kiekintveld, “Optimal network security hardening using attack graph games,” in *Proc. 24th International Joint Conference on Artificial Intelligence (IJCAI)*, 2015.
10. E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
11. P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
12. J. Y. Yen, “Finding the K shortest loopless paths in a network,” *Management Science*, vol. 17, no. 11, pp. 712–716, 1971, doi: 10.1287/mnsc.17.11.712.
13. A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. 7th Python in Science Conference (SciPy 2008)*, Pasadena, CA, USA, 2008, pp. 11–15.
14. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
15. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

16. G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, “An analysis of approximations for maximizing submodular set functions—I,” *Mathematical Programming*, vol. 14, pp. 265–294, 1978.
17. J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: University of Michigan Press, 1975.
18. D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.
19. R. E. Fikes and N. J. Nilsson, “STRIPS: A new approach to the application of theorem proving to problem solving,” *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971.
20. FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com
21. React documentation. [Online]. Available: https://react.dev
22. Render documentation, “Blueprint specification.” [Online]. Available: https://render.com/docs/blueprint-spec
23. GitHub documentation, “Using custom workflows with GitHub Pages.” [Online]. Available: https://docs.github.com/en/pages

References [20]–[23] support the implementation and deployment descriptions in Sections 5 and 10.

10. DEPLOYMENT

The application is publicly deployed using only free services: GitHub Pages for the front end, and Render's free web-service and free PostgreSQL plans for the back end [22], [23]. Table 10.1 summarises the deployment, and Fig. 10.1 shows the live site as deployment evidence.

In simple terms, the website that users open is stored on GitHub Pages. Each time it needs a calculation, it sends a request over the Internet to the back-end server on Render, which runs the algorithms and replies. The database also runs on Render and stores saved networks and experiment results. Anyone with the link can use the application from a browser without installing anything.

Table 10.1 Deployment Summary

Component	Service (plan)	Public URL	Verification
Front end	GitHub Pages (free)	`https://sid07-pro.github.io/cyber-deception-ai/`	Loads; API connected; simulation runs
Back-end API	Render web service (free)	`https://cyber-deception-api.onrender.com`	/api/health returns status ok
Database	Render PostgreSQL (free)	Internal to Render	Health reports database connected
Source code	GitHub	`https://github.com/sid07-pro/cyber-deception-ai`	Branch main

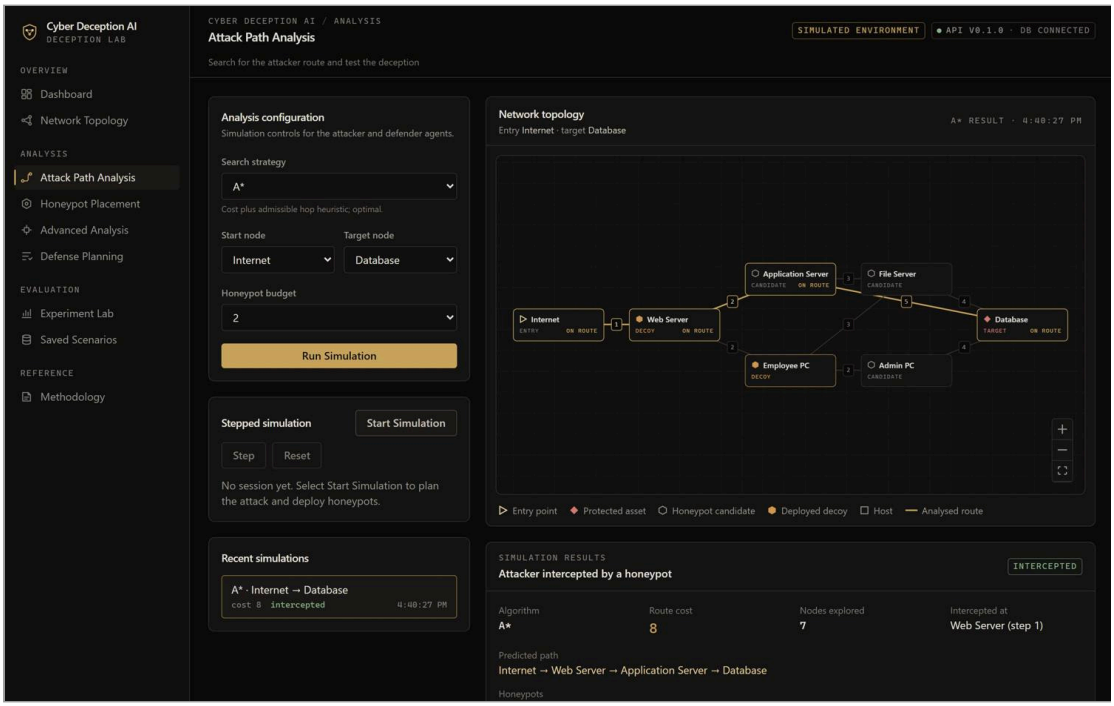


Fig. 10.1 Live Deployment on GitHub Pages

The live API health check returned the following response:

```
GET https://cyber-deception-api.onrender.com/api/health
{"status":"ok","service":"cyber-deception-ai","version":"0.1.0",
 "environment":"production","database":"connected"}
```

10.1 Deployment Process

- **Back end:** the repository's `render.yaml` blueprint creates the free web service (root `backend`, start command `uvicorn app.main:app --host 0.0.0.0 --port $PORT`, health check `/api/health`) and the free PostgreSQL database. Render injects `DATABASE_URL`.
- **CORS:** `ALLOWED_ORIGINS` is set to `https://sid07-pro.github.io`, so in production only the deployed front end may call the API.
- **Front end:** the GitHub Actions workflow `.github/workflows/pages.yml` builds the React app with `VITE_API_BASE_URL` set to the Render API and `VITE_BASE_PATH=/cyber-deception-ai/`, then publishes it to GitHub Pages on every push to `main`.

10.2 Local Execution

```
# Back end (Python 3.12)
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Front end (Node.js 20+), in a new terminal
cd frontend
npm install
npm run dev            # http://localhost:5173

# Or the full stack with Docker Compose (set POSTGRES_PASSWORD in .env
first)
docker compose up -d --build
```

10.3 Testing

```
cd backend  && .venv\Scripts\python -m pytest -q
cd frontend && npx tsc -b --noEmit && npx vitest run && npm run build
```

10.4 Free-Tier Notes and Security

- No paid plan is used and no payment method is attached.
- The free Render web service sleeps after about 15 minutes of inactivity, so the first request afterwards can take up to a minute.

- Render’s free PostgreSQL database is time-limited (about 30 days) and must then be recreated. Simulations do not depend on it.
- The system is simulation-only, with no real hosts, scanning or exploits. The API validates all input and restricts CORS, but it has no user authentication.
- No secrets are committed; database credentials are supplied by Render as environment variables.